TOROS UNIVERSITY
ENGINEERING FACULTY
DEPARTMENT OF SOFTWARE ENGINEERING

SWE403 — GRADUATION THESIS

Design and Implementation of a Linux Process Manipulation and Runtime Analysis Tool

By

21506008
Ulaş Tan Ersoyak

2025

Abstract

This thesis investigates how a running program on Linux can be inspected and modified from the outside, and it does so by building a tool—*Process Processor* (`pp`)—from first principles rather than by using an existing debugger. The central research question is whether the three capabilities usually provided by separate, mature tools—memory inspection, runtime code modification, and symbol-level analysis—can be reconstructed on top of only the primitive interfaces the kernel exposes (`ptrace`, the `/proc` filesystem, the cross-process memory system calls, and the ELF format), and what the attempt reveals about the boundary the operating system enforces between processes. A recurring mechanism underlies every modification feature: because a process may only allocate or re-protect its *own* address space, `pp` makes the target execute the required system call on its own behalf by hijacking a thread's registers, planting a small instruction stub, and regaining control through a breakpoint. The tool implements this mechanism for memory allocation (`mmap`), permission changes (`mprotect`), shared-library injection (`dlopen`), and function hooking. The evaluation shows that the inspection and injection features are reliable, while function hooking succeeds only for self-contained functions: the experiment exposes relocation and stack-alignment problems that are intrinsic to code injection rather than incidental to the implementation. The work is therefore presented as a study of the mechanisms and their limits, not as a production debugger.

Keywords: Linux, process manipulation, runtime analysis, `ptrace`, `/proc`, ELF, memory management, code injection, function hooking, disassembly.

Contents

1 Introduction **2**
- 1.1 Motivation . . . 2
- 1.2 Problem Statement . . . 2
- 1.3 Research Questions . . . 2
- 1.4 Contributions . . . 2
- 1.5 Organisation . . . 2

2 Background and Related Work **2**
- 2.1 Interfaces for External Process Access . . . 2
- 2.2 Existing Tools and Positioning . . . 3

3 Design and Methodology **3**
- 3.1 Design Goals . . . 3
- 3.2 Architecture . . . 3
- 3.3 The Uniform Injection Mechanism . . . 3
- 3.4 Modification Strategies Built on the Mechanism . . . 4

4 Implementation **5**
- 4.1 Memory Inspection . . . 5
- 4.2 Pattern Search and Replacement . . . 5
- 4.3 Symbol Resolution . . . 5
- 4.4 Disassembly . . . 5

5 Evaluation and Discussion **5**
- 5.1 What Works, and How It Was Confirmed . . . 5
- 5.2 Where It Breaks: Function Hooking . . . 5
- 5.3 Threats to Validity . . . 6

6 Conclusion **6**

1 Introduction

1.1 Motivation

Understanding how a program behaves while it is running is a recurring need in debugging, security analysis, and systems education. The state that matters at run time—the contents of memory, the layout of code, the register context of each thread—is not visible in the source and only partially visible in the binary. Linux exposes this state, and the means to change it, through a small set of low-level interfaces: the `/proc` filesystem, the `ptrace` system call, the cross-process memory calls `process_vm_readv` and `process_vm_writev`, and the ELF format that describes every executable and shared object. Mature tools such as debuggers and dynamic instrumentation frameworks are built on exactly these interfaces, but they present them behind large abstractions that hide the underlying mechanics. The mechanics themselves—how one process is allowed to read another's memory, why it cannot simply change that memory's permissions, how a function can be redirected while the program runs—are what this thesis sets out to examine.

1.2 Problem Statement

A debugger or an instrumentation framework answers the question "what is this program doing?" but does not, by its nature, explain *how* the answer is obtained. The operating system deliberately isolates processes from one another; any tool that inspects or alters a foreign process must work within, and occasionally around, that isolation. The problem addressed here is to make those boundary-crossing mechanisms explicit by implementing them directly, and to establish where they hold and where they break down. Rather than consume an existing abstraction, this work reconstructs one, so that each capability corresponds to a concrete, inspectable use of a kernel interface.

1.3 Research Questions

The thesis is organised around three questions:

1. **RQ1.** Can memory inspection, symbol-level program analysis, and runtime code modification be built on the primitive Linux interfaces alone, without relying on an existing debugger?
2. **RQ2.** Given that a process may modify only its own address space, by what mechanism can an external tool cause a target to allocate memory, change permissions, and load code on its own behalf—and is that mechanism uniform across these operations?
3. **RQ3.** Where are the practical limits of external code modification, and are those limits a property of the implementation or of the technique itself?

1.4 Contributions

The thesis makes the following contributions:

- A single tool, `pp`, that unifies memory inspection, symbol resolution, disassembly, and runtime code modification over a common model of processes and memory regions, implemented directly on kernel interfaces.
- An account of a *uniform injection mechanism*—register hijack, instruction-stub insertion, and breakpoint return—shown to serve `mmap`, `mprotect`, and `dlopen` alike.
- A qualitative evaluation that separates the capabilities which work reliably from function hooking, which does not, and attributes the difference to relocation and stack alignment rather than to defects in the code.

1.5 Organisation

Section 2 reviews the interfaces the tool depends on and positions `pp` against existing tools. Section 3 describes the design and the methodology, centred on the injection mechanism. Section 4 details the implementation of the inspection and modification features. Section 5 evaluates the result and discusses the limits that emerged. Section 6 returns to the research questions.

2 Background and Related Work

2.1 Interfaces for External Process Access

Only the facts that the rest of the thesis relies on are summarised here; the manual pages are the authoritative reference.

The `/proc` filesystem presents kernel state as files. Each process has a directory `/proc/[pid]`; `pp` reads `comm` for the process name, `maps` for the list of mapped memory regions (each with an address range, permission bits `rwxp`, and an optional backing file), `statm` for resident-memory accounting, `exe` for the executable path, and the `task/` subdirectory to enumerate threads.

Cross-process memory access. `process_vm_readv` and `process_vm_writev` copy data directly between the address spaces of two processes without trapping through

`/proc/[pid]/mem`, subject to the same permission checks. `pp` uses them for all bulk reads and writes of target memory.

Process control. `ptrace` lets a tracer attach to a thread, stop it, read and write its registers (`PTRACE_GETREGS` / `PTRACE_SETREGS`), read and write individual words of its memory (`PTRACE_PEEKTEXT` / `PTRACE_POKETEXT`), and resume it. This register- and word-level control is what makes the injection mechanism of Section 3 possible.

The isolation boundary. A process may allocate memory (`mmap`) or change the protection of a region (`mprotect`) only within its own address space; there is no system call that lets one process do this to another. This single restriction is the reason the tool cannot simply "allocate memory in the target" and instead must make the target perform the call itself—the pivot on which the design turns.

ELF. Executables and shared objects carry symbol tables (`.symtab`/`.dynsym` with their string tables) that map function names to addresses, and program headers whose first `PT_LOAD` segment fixes the load bias needed to translate a symbol's value into a runtime address. `pp` parses these structures directly to resolve functions.

2.2 Existing Tools and Positioning

Several categories of tool already perform parts of what `pp` does. General-purpose debuggers such as GDB are built on `ptrace` and provide breakpoints, memory inspection, and symbol resolution, but as general systems they hide these mechanisms behind a command language and a large runtime. Tracers such as `strace` and `ltrace` observe system and library calls without modifying behaviour. The loader-level technique of `LD_PRELOAD` interposes replacement functions, but only at process start and only for dynamically resolved symbols. Dynamic instrumentation frameworks, ranging from academic systems such as Pin [1], which rewrites a program's instruction stream through a just-in-time compiler, to production tools such as Frida, inject substantial machinery into the target and expose a high-level API for hooking and tracing. Memory scanners and "trainers" such as `scanmem` and Cheat Engine specialise in searching and editing live memory values but do not perform code injection or symbol analysis.

`pp` is not intended to compete with any of these. Its distinguishing choice is pedagogical: it reconstructs inspection, analysis, and modification *from the primitives*, in one small codebase, so that each feature is a visible use of a kernel interface rather than a call into an abstraction. The uniform injection mechanism that this exercise surfaces (Section 3) is the technical core; the breadth of commands is secondary.

3 Design and Methodology

3.1 Design Goals

The tool was built to satisfy three goals derived from the research questions: every feature should rest on a primitive interface rather than on another tool; the modification features should share one mechanism rather than each inventing its own; and the model of a process and its memory should be uniform enough that inspection, analysis, and modification operate on the same abstractions. The development method was iterative and exploratory—each feature was implemented against a live process and validated by observation—which is appropriate for a study whose aim is to understand mechanisms and expose their limits rather than to meet a fixed specification.

3.2 Architecture

`pp` is organised into a small number of modules with clear responsibilities. A `process` abstraction represents a target and owns everything derivable from `/proc` and the on-disk binary: its name, memory regions, threads, base address, and resolved functions. A `memory_region` abstraction models one mapped range—its bounds, permissions, and backing name—and is the common currency of every memory operation. A `debugger` abstraction holds an active `ptrace` attachment to every thread of the target and implements the operations that require control of execution: reading and writing registers, allocating memory, changing permissions, injecting a library, and hooking a function. Supporting modules provide disassembly (a thin wrapper over the Capstone framework), the command-line interface, and utilities for ELF and file handling. The separation lets the same region and process model serve a read-only inspection command and a destructive modification command without special-casing.

3.3 The Uniform Injection Mechanism

The isolation boundary described in Section 2 means that operations like "allocate a page in the target" or "make this region executable" cannot be issued from the tool. The design answer, and the single most important idea in the implementation, is to make the *target* issue the call while the tool choreographs it. The procedure, shown in Figure 1, is identical for every such operation:

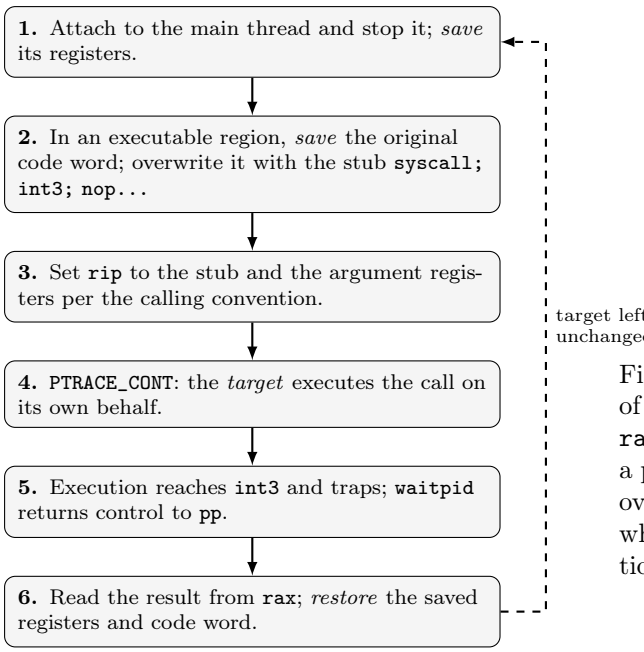


Figure 1: The uniform injection mechanism. Steps 1–3 and 6 are performed by `pp` through `ptrace`; steps 4–5 run inside the target. The same cycle serves `mmap`, `mprotect`, and `dlopen`, differing only in the stub and the register setup.

1. Attach to and stop the target's main thread; save its registers.
2. Locate an executable region and save the machine-code word at its start.
3. Overwrite that word with a short stub: the instruction that performs the operation, followed by an `int3` breakpoint (`0xCC`) and `nop` padding.
4. Set the registers so that the instruction pointer points at the stub and the argument registers follow the relevant calling convention.
5. Resume the thread and `waitpid` for it to hit the breakpoint.
6. Read the result from the register file, then restore both the saved registers and the saved code word, leaving the target as it was found.

For a system call the stub is simply `syscall; int3`, assembled as the word `0x90909090CC050F` (`0F 05` followed by `CC` and padding), with `rax` set to the syscall number and `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9` set to its arguments. Allocation sets `rax` to the `mmap` number and lets the kernel choose an address; a permission change sets it to the `mprotect` number with the target region's bounds and the desired protection. The breakpoint is what returns control: it stops the target and hands the tool a defined point at which to read the return value and undo the modification. Because the mechanism is uniform, adding a new privileged operation is a matter of choosing the stub and the register setup rather than designing a new technique.

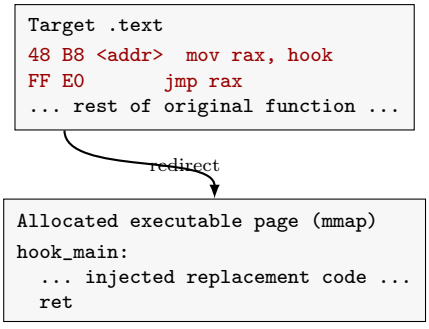


Figure 2: Function hooking. The first bytes of the target function are overwritten with `mov rax, <hook>; jmp rax`, redirecting execution into a page holding the extracted `hook_main` code. The overwritten prologue bytes are not relocated elsewhere, which is the source of the limitations in Section 5.

3.4 Modification Strategies Built on the Mechanism

Two higher-level features extend the mechanism beyond a single system call.

Shared-library injection causes the target to call `dlopen` on a path the tool has written into target memory. Here the stub is a `call` through a register holding the resolved `dlopen` address rather than a `syscall`, and a freshly allocated region is used as a clean stack so that the call has somewhere to push its return address. Using the target's own dynamic linker means the injected library is loaded exactly as if it had been present at start-up, including the execution of its constructors, and can be unloaded again—making this the least invasive modification strategy.

Function hooking redirects a function to replacement code. The replacement is written as an ordinary C++ source file exposing a `hook_main` symbol; `pp` compiles it, extracts just that function's bytes from the resulting ELF, allocates an executable page in the target, writes the code there, and finally overwrites the prologue of the target function with a sequence that loads the replacement's address and jumps to it (Figure 2). This is the classic in-place interception approach popularised by Detours [2]; it is the most invasive strategy and, as Section 5 shows, the one whose assumptions are most easily violated.

4 Implementation

This section records the implementation decisions that are not obvious from the mechanism itself. It deliberately omits the catalogue of command-line commands, which is operational detail rather than a research result.

4.1 Memory Inspection

Reads and writes of target memory go through `process_vm_readv` and `process_vm_writev`, wrapped in a single templated operation that works uniformly whether the caller addresses the process or an individual thread. Memory maps are produced by parsing `/proc/[pid]/maps` into `memory_region` objects; memory statistics are aggregated from region sizes and permission bits, with anonymous regions identified by the absence of a backing name. Thread inspection enumerates `/proc/[pid]/task` and reads each thread's register context through `ptrace`.

4.2 Pattern Search and Replacement

Searching scans every readable region for a byte or ASCII pattern and reports each match's true address by adding the region base to the in-buffer offset. Replacement operates only on regions that are both readable and writable; it advances past each replacement before continuing, so that a replacement which itself contains the search pattern is not matched again, and it reports the exact number of substitutions performed rather than an estimate.

4.3 Symbol Resolution

Function resolution parses the target binary's ELF symbol tables and translates a symbol's value into a runtime address using the load bias taken from the first `PT_LOAD` segment, so that both the whole-program listing and the lookup of a single function agree on the same address arithmetic. Lookups are matched by exact name and cached, and C++ names can be demangled for readability.

4.4 Disassembly

Disassembly reads a region of target memory and decodes it with Capstone, emitting address, mnemonic, operands, and instruction length. Beyond analysis, disassembly serves as a verification tool: after a hook is installed, decoding the target's prologue confirms that the redirecting jump was written correctly.

5 Evaluation and Discussion

5.1 What Works, and How It Was Confirmed

The inspection features—maps, statistics, thread and register state, symbol listing, and disassembly—were validated by comparing `pp`'s output against known references: `/proc` contents read independently, and symbol addresses cross-checked against the disassembly at those addresses. Cross-process read and write were confirmed by writing a value and reading it back, and by observing the effect of pattern replacement on a running target. The injection mechanism of Section 3.3 was confirmed for both `mmap` and `mprotect`: after each operation the target resumes normally, which demonstrates that the save/restore of registers and the original code word is correct, and the returned region is usable. Library injection via `dlopen` loads a shared object into the target and runs its constructors. These results answer RQ1 affirmatively—the inspection and analysis stack can be rebuilt on the primitives alone—and RQ2—the same register-hijack mechanism serves allocation, protection, and library loading without modification.

5.2 Where It Breaks: Function Hooking

Function hooking is reliable only for simple, self-contained functions. The limits it runs into are instructive and bear directly on RQ3:

- **Relocation.** The replacement code is extracted as raw bytes and placed at an address unrelated to where it was compiled. Any position-dependent reference—a call to another function, a reference to global data—is left pointing at the wrong place, because `pp` performs no relocation of the injected code. Only functions that reference nothing outside themselves survive the move intact.
- **Stack alignment.** The x86-64 ABI requires a particular stack alignment at function entry. Injected code that is entered through a hand-written jump, rather than a normal call, can violate this and crash in library routines that assume it.
- **Prologue overwriting.** Redirecting a function by overwriting its first bytes assumes the prologue is at least as long as the jump sequence and that no other instruction jumps into the middle of what was overwritten. Short functions and jump targets inside the prologue break this assumption.

The important point for the thesis is that these failures are properties of the *technique*, not bugs: robust hooking requires a full relocating loader and disassembly-aware prologue analysis—the trampoline construction that Detours [2] relocates the displaced instructions into—which are substantial systems in their own right. The experiment therefore delimits the boundary it set out to find—inspection and self-contained injection lie inside the reach of the primitives, while general code rewriting lies beyond what the primitives alone can safely support.

5.3 Threats to Validity

The evaluation is qualitative and conducted on x86-64 Linux with a small set of target programs; the tool is architecture- and platform-specific by construction, and the hooking results in particular depend on the compiler and optimisation level used to produce both the target and the replacement. The conclusions about hooking generalise to the technique, but the exact set of functions that hook successfully will vary between binaries.

6 Conclusion

This thesis asked whether the capabilities of a debugger could be rebuilt on the Linux primitives alone, by what mechanism an external tool can drive privileged operations in a process it does not own, and where external code modification reaches its limit. Building `pp` answered all three. The inspection and analysis layer—memory maps, cross-process read and write, pattern operations, symbol resolution, and disassembly—was reconstructed directly on `/proc`, the cross-process memory calls, `ptrace`, and the ELF format, confirming RQ1. The modification layer was unified under a single mechanism: stopping a thread, hijacking its registers, planting a stub, and returning through a breakpoint, which served `mmap`, `mprotect`, and `dlopen` without change, confirming RQ2 and showing that the isolation boundary can be crossed cleanly when the target is made to act on its own behalf. The answer to RQ3 is the most informative: the boundary of *safe* external modification falls between library injection, which works because it reuses the target's own loader, and arbitrary code injection, which does not, because faithfully relocating code and preserving the ABI amount to reimplementing the loader the technique was trying to avoid.

The broader lesson is that the operating system's isolation of processes is not a wall but a set of well-defined gates: memory can be read and written across the boundary, and a process can be made to reconfigure itself, but code cannot be transplanted into it without carrying along the machinery that made that code valid. `pp` remains an experimental, educational tool rather than a production debugger, and the honest demarcation of what it cannot do is as much a result as what it can.

Future work follows directly from the limits found rather than from a wish list: a relocating loader and a disassembly-driven trampoline would turn hooking from a special case into a general capability; broadening symbol resolution to handle complex C++ and stripped binaries would widen the analysis layer; and porting the register conventions would lift the x86-64 restriction. Each of these extends a boundary this work was able to locate precisely because it built the mechanisms itself.

References

[1] C.-K. Luk, R. Cohn, R. Muth, H. Patil, A. Klauser, G. Lowney, S. Wallace, V. J. Reddi, and K. Hazelwood, "Pin: Building Customized Program Analysis Tools with Dynamic Instrumentation," in *Proc. ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, 2005, pp. 190–200.

[2] G. Hunt and D. Brubacher, "Detours: Binary Interception of Win32 Functions," in *Proc. 3rd USENIX Windows NT Symposium*, 1999, pp. 135–143.